

## АНАЛИЗ ТЕХНИЧЕСКОГО ЗАДАНИЯ

### 1. Что нужно сделать (суть проекта)

Разработать программу «Layout Architect» — интерактивную среду для сравнения алгоритмов процедурной генерации уровней (BSP и Cellular Automata). Программа должна:

Генерировать уровни двумя разными алгоритмами

Позволять настраивать параметры (seed, размеры, % стен и т.д.)

Показывать карту на экране

Считать метрики (связанность, количество комнат, коэффициент ветвления, площадь)

Иметь режим, где можно походить по уровню персонажем

Уметь сохранять статистику в CSV

## 2. Какие технологии выбрал

| Что          | Выбор              | Почему                              |
|--------------|--------------------|-------------------------------------|
| Движок       | Godot 4.x          | Легкий, бесплатный, отличный для 2D |
| Язык         | GDScript           | Встроен в Godot, простой, быстрый   |
| Визуализация | TileMap            | Удобно для сеточных уровней         |
| Интерфейс    | Control-узлы Godot | Всё нужное уже есть                 |
| Экспорт      | CSV                | Открывается в Excel/R/Python        |

### **3. Из чего будет состоять программа (модули)**

Генерация

-BSP -CA Seed

Визуализация

-TileMap -Камера -Спрайты

Аналитика

-BFS -Комнаты -Ветвление

Симуляция

- Игрок- Коллизии- Враг

Ввод/Вывод

- UI- CSV export

Как модули связаны:

Генерация создает карту → отдает Визуализации и Аналитике

Визуализация показывает карту → в ней работает Симуляция

Аналитика считает метрики → отдает в UI и на экспорт

UI управляет всем и показывает результаты

## 4. Разбор функций по пунктам

### 4.1. Управление генерацией

| Требование         | Как буду делать                       | Сложность |
|--------------------|---------------------------------------|-----------|
| Выбор алгоритма    | Выпадающий список (OptionButton)      | Легко     |
| Ввод seed          | Поле ввода + кнопка Random            | Легко     |
| Настройки BSP      | Поля для размеров и мин/макс комнат   | Легко     |
| Настройки СА       | Поля для размеров, % стен, итераций   | Легко     |
| Кнопка генерации   | Обработчик нажатия, вызов алгоритма   | Легко     |
| Время $\leq 3$ сек | Оптимизация кода (линейные алгоритмы) | Средне    |

### 4.2. Визуализация

| Требование             | Как буду делать           | Сложность |
|------------------------|---------------------------|-----------|
| Разные типы клеток     | TileMap с разными тайлами | Легко     |
| Выделить старт/выход   | Спрайты поверх карты      | Легко     |
| Перемещение камеры     | Camera2D с drag-скроллом  | Легко     |
| Показать алгоритм/seed | Текстовые метки на экране | Легко     |

### 4.3. Аналитика

| Требование              | Как буду делать               | Сложность |
|-------------------------|-------------------------------|-----------|
| Авторасчет после ген-ии | Вызов функций после генерации | Легко     |
| Связанность             | BFS от старта                 | Средне    |
| Количество комнат       | Поиск связанных компонент     | Средне    |
| Коэф. ветвления         | Среднее число соседей         | Средне    |
| Общая площадь           | Простой подсчет               | Легко     |
| Панель метрик           | Текстовые метки, обновление   | Легко     |

#### 4.4. Симуляция

| Требование           | Как буду делать                     | Сложность |
|----------------------|-------------------------------------|-----------|
| Переключение режимов | По клавише Tab                      | Легко     |
| Управление           | WASD/стрелки,<br>CharacterBody2D    | Средне    |
| Коллизии             | TileMap с коллизиями                | Средне    |
| Враг                 | Простой объект, при касании респаун | Легко     |

#### 4.5. Эксперименты и данные

| Требование         | Как буду делать                   | Сложность |
|--------------------|-----------------------------------|-----------|
| Пакетная генерация | Цикл по seed'ам                   | Средне    |
| Экспорт в CSV      | Формирование строк, запись в файл | Средне    |

## 5. Алгоритмы генерации (как работают)

### BSP (Binary Space Partitioning)

1. Берем прямоугольник (весь уровень)
2. Если он слишком маленький → это комната
3. Иначе режем пополам (вертикально или горизонтально)
4. Для каждой половинки повторяем шаги 1-3
5. Соединяем получившиеся комнаты коридорами

Сложность:  $O(n \log n)$  — быстро

### Cellular Automata

1. Заполняем сетку случайно (стены/пол)
2. Для каждой клетки считаем соседей-стен
3. Если клетка стена и соседей мало → становится полом
4. Если клетка пол и соседей много → становится стеной
5. Повторяем шаги 2-4 несколько раз

Сложность:  $O(n \times \text{итерации})$  — для  $100 \times 100$  и 10 итераций ~100к операций

## 6. Метрики

Связанность

Старт → BFS (поиск в ширину) по проходимым клеткам

Считаем, сколько клеток посетили

Делим на общее число проходимых клеток

Умножаем на 100%

Количество комнат

Проходим по всем клеткам

Если нашли пол и не посещали раньше → новая комната

Обходим всю комнату (BFS/DFS)

Считаем количество таких комнат

Коэффициент ветвления

Для каждой проходимой клетки считаем соседей-пол

Суммируем

Делим на количество проходимых клеток

Площадь

Просто считаем все клетки с типом "пол"